ISO/IEC JTC 1/SC 22/WG14
WG 21, SG 22

N2695 v2
P2306R1

2021-3-28

Type-generic lambdas
proposal for C23

Jens Gustedt
INRIA and ICube, Université de Strasbourg, France

For the lambda expressions that were introduced in N2694, we propose the addition of **auto** parameters that can be completed by the arguments (in a function call) or by the parameter types of target function pointer (in a conversion).

Changes:

v2/R1.

— make primary expressions transparent for lambda expression operands
— force types to be the same when a tg lambda is converted
— specify with syntax verifications are necessary for lamda expressions in void expressions

I. MOTIVATION

This paper is fully motivated in N2693, namely for the improvement of type-generic programming in C. For a simple motivation of the feature compared to simple lambdas see for example the MAXIMUM macro in the proposed text, 6.5.2.6 p17.

II. DESIGN CHOICES

We chose to follow C++ syntax and semantic as close a possible.

II.1. Permissible contexts for type-generic lambdas

It is the intent of this paper, to allow a value of a type-generic lambda type only in a context where it will be completed, either by the arguments of a function call or by the parameter types of a target function pointer to which a type-generic function literal is converted. This is to ensure that compilers that implement this feature have to do no lookahead or pre-compilation of code snippets with a lot of unknown types.

This is achieved by integrating types of type-generic lambdas into the terminology of the standard as being incomplete types. Thereby it is not possible to define objects of such a type. Because lambdas can only be declared in definitions by type inference, effectively such lambdas cannot even be declared.

By these properties, the only possibility to specify a type-generic lambda that is re-usable at different places of a source is *textual*, in particular by defining function-like macros. This restriction is a deliberate choice for this proposal, here. If in a later phase (probably C26) WG14 would also want to add objects of type-generic lambda type to the language or adopt C++'s template functions, this could easily be achieved on top of what is done here.

II.2. Parameter type inference

Parameter type inference only leaves a design choice for array and function parameters. To be in line with traditional function declarations, we extend the possibility of type inference

to such types and specify that these are to be re-written to pointers to form a valid function prototype.

III. SYNTAX AND TERMINOLOGY

For all proposed wording see Section VII.

Syntax considerations for this feature are straight forward; we just have to allow the **auto** feature to extend to the parameters of lambdas, 6.7.6.3.

In terms of terminology, we introduce the terms *incomplete lambda type* (6.2.5 p20) and *type-generic lambda* (6.5.2.6 p9).

IV. SEMANTICS

The principal semantics of type-generic lambdas are described within three paragraphs.

— Paragraph 6.2.5.6 p9 specifies the possible use of type-generic lambdas.
— Paragraph 6.2.5.6 p10 provides the rules for the completion of such a lambda in a function call.
— An insertion into 6.3.2.1 p5 describes the mechanism for conversions of type-generic function literals to function pointers.

V. CONSTRAINTS AND REQUIREMENTS

This proposal constrains the possible uses of type-generic lambdas even further than for simple lambdas, namely essentially to function calls and conversions to pointer-types. Even though it would have been possible to formulate such a requirement as a constraint, we chose not to do so because this might be an area for implementations to extend the C standard and to implement some template feature for lambda values. Forcing them to diagnose such constructs would be counter-productive and hinder progress in that area.

The only constraint that this proposal includes is in 6.5.2.6 p6, namely that a type-generic lambda that is used in a conversion to a function pointer must have a return type that is compatible to the one of the target function pointer type.

VI. QUESTIONS FOR WG14

(1) Does WG14 want type-generic lambdas for C23 along the lines of N2695?
(2) Does WG14 want to integrate the changes as specified in N2695 into C23?

References

Jens Gustedt. 2021a. *Function literals and value closures*. Technical Report N2694. ISO. available at http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2694.pdf.

Jens Gustedt. 2021b. *Improve type generic programming*. Technical Report N2693. ISO. available at http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2693.pdf.

Jens Gustedt. 2021c. *Lvalue closures*. Technical Report N2696. ISO. available at http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2696.pdf.

Jens Gustedt. 2021d. *Type-generic lambdas*. Technical Report N2695. ISO. available at http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2695.pdf.

Jens Gustedt. 2021e. *Type inference for variable definitions and function return*. Technical Report N2697. ISO. available at http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2697.pdf.

VII. PROPOSED WORDING

The proposed text is given as diff against N2694.

— Additions to the text are marked as shown.
— Deletions of text are marked as ~~shown~~.

20 Any number of *derived types* can be constructed from the object and function types, as follows:

— An *array type* describes a contiguously allocated nonempty set of objects with a particular member object type, called the *element type*. The element type shall be complete whenever the array type is specified. Array types are characterized by their element type and by the number of elements in the array. An array type is said to be derived from its element type, and if its element type is *T*, the array type is sometimes called "array of *T*". The construction of an array type from an element type is called "array type derivation".

— A *structure type* describes a sequentially allocated nonempty set of member objects (and, in certain circumstances, an incomplete array), each of which has an optionally specified name and possibly distinct type.

— A *union type* describes an overlapping nonempty set of member objects, each of which has an optionally specified name and possibly distinct type.

— A *function type* describes a function with specified return type. A function type is characterized by its return type and the number and types of its parameters. A function type is said to be derived from its return type, and if its return type is *T*, the function type is sometimes called "function returning *T*". The construction of a function type from a return type is called "function type derivation".

— A *lambda type* is ~~a complete~~ an object type that describes the value of a lambda expression. A complete lambda type is characterized but not determined by a return type that is inferred from the function body of the lambda expression, and by the number, order, and type of parameters that are expected for function calls~~.~~ ; ~~The~~ the function type that has the same return type and list of parameter types as the lambda is called the *prototype* of the lambda. A lambda type has no syntax derivation.[50] Objects of such a type shall only be defined as a capture (of another lambda expression) or by an underspecified declaration for which the lambda type is inferred.[51] An object of lambda type shall only be modified by simple assignment (6.5.16.1). A lambda expression that has underspecified parameters has an incomplete lambda type that can be completed by function call arguments, or, if it has no captures, in a conversion to a function pointer.

— A *pointer type* may be derived from a function type or an object type, called the *referenced type*. A pointer type describes an object whose value provides a reference to an entity of the referenced type. A pointer type derived from the referenced type *T* is sometimes called "pointer to *T*". The construction of a pointer type from a referenced type is called "pointer type derivation". A pointer type is a complete object type.

— An *atomic type* describes the type designated by the construct **_Atomic**(*type-name*). (Atomic types are a conditional feature that implementations need not support; see 6.10.8.3.)

These methods of constructing derived types can be applied recursively.

21 Arithmetic types and pointer types are collectively called *scalar types*. Array and structure types are collectively called *aggregate types*.[52]

22 An array type of unknown size is an incomplete type. It is completed, for an identifier of that type, by specifying the size in a later declaration (with internal or external linkage). A structure or union type of unknown content (as described in 6.7.2.3) is an incomplete type. It is completed, for all declarations of that type, by declaring the same structure or union tag with its defining content later in the same scope.

23 A type has *known constant size* if the type is not incomplete and is not a variable length array type.

[50] Not even a **typeof** type specifier with lambda type can be formed. So there is no syntax to make a lambda type a choice in a generic selection other than **default**

[51] Another possibility to create an object that has an effective lambda type is to copy a lambda value into allocated storage via simple assignment.

[52] Note that aggregate type does not include union type because an object with union type can only contain one member at a time.

the object. A *modifiable lvalue* is an lvalue that does not have array type, does not have an incomplete type, does not have a const-qualified type, and if it is a structure or union, does not have any member (including, recursively, any member or element of all contained aggregates or unions) with a const-qualified type.

2 Except when it is the operand of the **typeof** specifier, the **sizeof** operator, the unary & operator, the `++` operator, the `--` operator, or the left operand of the `.` operator or an assignment operator, an lvalue that does not have array type is converted to the value stored in the designated object (and is no longer an lvalue); this is called *lvalue conversion*. If the lvalue has qualified type, the value has the unqualified version of the type of the lvalue; additionally, if the lvalue has atomic type, the value has the non-atomic version of the type of the lvalue; otherwise, the value has the type of the lvalue. If the lvalue has an incomplete type and does not have array type, the behavior is undefined. If the lvalue designates an object of automatic storage duration that could have been declared with the **register** storage class (never had its address taken), and that object is uninitialized (not declared with an initializer and no assignment to it has been performed prior to use), the behavior is undefined.

3 Except when it is the operand of the **typeof** specifier, the unary **sizeof** operator, or the unary & operator, or is a string literal used to initialize an array, an expression that has type "array of *type*" is converted to an expression with type "pointer to *type*" that points to the initial element of the array object and is not an lvalue. If the array object has register storage class, the behavior is undefined.

4 A *function designator* is an expression that has function type. Except when it is the operand of the **typeof** specifier, the **sizeof** operator,[72] or the unary & operator, a function designator with type "function returning *type*" is converted to an expression that has type "pointer to function returning *type*".

5 ~~Closures~~ Other than specified in the following, lambda types shall not be converted to any other object type. A complete function literal with a type "lambda with prototype `P`" can be converted implicitly or explicitly to an expression that has type "pointer to `Q`", where `Q` is a function type that is compatible with `P`.[73] For a type-generic function literal expression, types of underspecified parameters shall first be completed according to the parameters of the target prototype, that is, for each underspecified parameter there shall be a type specifier of a unique type as described in 6.7.11 such that the adjusted parameter type is the same as the adjusted parameter type of the target function type; after that, the prototype `P` of the thus completed lambda expression shall be the target prototype `Q`.[74] The function pointer value behaves as if a function `F` of type `P` with internal linkage, a unique name, and the same ~~parameter list and~~ function body as for λ, where uses of identifiers from an outer scope in expressions that are not evaluated are replaced by proper types or values, had been defined in the translation unit ~~,~~and the function pointer had been formed by function-to-pointer conversion of that function. The only ~~difference is~~ differences are that, if λ is not type-generic, the resulting function pointer is the same for the whole program execution whenever a conversion of λ is met[75] and that the function pointer needs not necessarily to be distinct from any other compatible function pointer that provides the same observable behavior.

Forward references: lambda expressions (6.5.2.6) address and indirection operators (6.5.3.2), assignment operators (6.5.16), common definitions `<stddef.h>` (7.19), **typeof** specifier 6.7.9, initialization (6.7.10), postfix increment and decrement operators (6.5.2.4), prefix increment and decrement operators (6.5.3.1), the **sizeof** and **_Alignof** operators (6.5.3.4), structure and union members (6.5.2.3)~~.~~, type inference (6.7.11).

---

[72] Because this conversion does not occur, the operand of the **sizeof** operator remains a function designator and violates the constraints in 6.5.3.4.

[73] It follows that lambdas of different type cannot be assigned to each other. Thus, in the conversion of a function literal to a function pointer, the prototype of the originating lambda expression can be assumed to be known, and a diagnostic can be issued if the prototypes do not aggree.

[74] Thus a specification of the target function pointer type in a conversion from a type-generic function literal expression that uses the `[*]` syntax for VM types is invalid.

[75] Thus a function literal that is not type-generic has properties that are similar to a function declared with **static** and **inline**. A possible implementation of the lambda type is to be the the function pointer type to which they convert.

— a type that is the signed or unsigned type corresponding to the effective type of the object,

— a type that is the signed or unsigned type corresponding to a qualified version of the effective type of the object,

— an aggregate or union type that includes one of the aforementioned types among its members (including, recursively, a member of a subaggregate or contained union), or

— a character type.

8 A floating expression may be *contracted*, that is, evaluated as though it were a single operation, thereby omitting rounding errors implied by the source code and the expression evaluation method.[100] The **FP_CONTRACT** pragma in `<math.h>` provides a way to disallow contracted expressions. Otherwise, whether and how expressions are contracted is implementation-defined.[101]

Forward references: the **FP_CONTRACT** pragma (7.12.2), copying functions (7.24.2).

6.5.1 Primary expressions

Syntax

1 *primary-expression:*
identifier
constant
string-literal
(*expression* **)**
generic-selection

Semantics

2 An identifier is a primary expression, provided it has been declared as designating an object (in which case it is an lvalue) or a function (in which case it is a function designator).[102]

3 A constant is a primary expression. Its type depends on its form and value, as detailed in 6.4.4.

4 A string literal is a primary expression. It is an lvalue with type as detailed in 6.4.5.

5 A parenthesized expression is a primary expression. Its type and value are identical to those of the unparenthesized expression. It is an lvalue, a function designator, a lambda expression, or a void expression if the unparenthesized expression is, respectively, an lvalue, a function designator, a lambda expression, or a void expression.

6 A generic selection is a primary expression. Its type and value depend on the selected generic association, as detailed in the following subclause.

Forward references: declarations (6.7).

6.5.1.1 Generic selection

Syntax

1 *generic-selection:*
_Generic (*assignment-expression* **,** *generic-assoc-list* **)**
generic-assoc-list:
generic-association
generic-assoc-list **,** *generic-association*
generic-association:
type-name **:** *assignment-expression*

[100] The intermediate operations in the contracted expression are evaluated as if to infinite range and precision, while the final operation is rounded to the format determined by the expression evaluation method. A contracted expression might also omit the raising of floating-point exceptions.

[101] This license is specifically intended to allow implementations to exploit fast machine instructions that combine multiple C operators. As contractions potentially undermine predictability, and can even decrease accuracy for containing expressions, their use needs to be well-defined and clearly documented.

[102] Thus, an undeclared identifier is a violation of the syntax.

`default :` *assignment-expression*

Constraints

2 A generic selection shall have no more than one **`default`** generic association. The type name in a generic association shall specify a complete object type other than a variably modified type. No two generic associations in the same generic selection shall specify compatible types. The type of the controlling expression is the type of the expression as if it had undergone an lvalue conversion,[103)] array to pointer conversion, or function to pointer conversion. That type shall be compatible with at most one of the types named in the generic association list. If a generic selection has no **`default`** generic association, its controlling expression shall have type compatible with exactly one of the types named in its generic association list.

Semantics

3 The controlling expression of a generic selection is not evaluated. If a generic selection has a generic association with a type name that is compatible with the type of the controlling expression, then the result expression of the generic selection is the expression in that generic association. Otherwise, the result expression of the generic selection is the expression in the **`default`** generic association. None of the expressions from any other generic association of the generic selection is evaluated.

4 The type and value of a generic selection are identical to those of its result expression. It is an lvalue, a function designator, a lambda expression, or a void expression if its result expression is, respectively, an lvalue, a function designator, a lambda expression, or a void expression. A generic selection that is the operand of a **`typeof`** specification behaves as if the selected assignment expression had been the operand.

5 **EXAMPLE** The **`cbrt`** type-generic macro could be implemented as follows:

```
#define cbrt(X) _Generic((X),                   \
                         long double: cbrtl,    \
                         default: cbrt,         \
                         float: cbrtf           \
                         )(X)
```

6.5.2 Postfix operators

Syntax

1 *postfix-expression:*
primary-expression
postfix-expression **`[`** *expression* **`]`**
postfix-expression **`(`** *argument-expression-list*$_{\text{opt}}$ **`)`**
postfix-expression **`.`** *identifier*
postfix-expression **`->`** *identifier*
postfix-expression **`++`**
postfix-expression **`-`**
`(` *type-name* **`)`** **`{`** *initializer-list* **`}`**
`(` *type-name* **`)`** **`{`** *initializer-list* **`,`** **`}`**
lambda-expression

argument-expression-list:
assignment-expression
argument-expression-list **`,`** *assignment-expression*

[103)] An lvalue conversion drops type qualifiers.

capture:
identifier

parameter-clause:
(*parameter-list*$_{\text{opt}}$ **)**

Constraints

2 A capture that is listed in the capture list is an *explicit capture*. If the capture clause is `[=]`, `id` is the name of an object with automatic storage duration in a surrounding scope that is not an array, `id` is used within the function body of the lambda without redeclaration and `id` is not a parameter, the effect is as if a capture list had been specified with `id` as a member. Such a capture is an *implicit capture*.

3 Captures without assignment expression shall be names of complete objects with automatic storage duration in a scope surrounding the lambda expression that do not have array type and that are visible at the point of evaluation of the lambda expression. An identifier shall appear at most once; either as an explicit capture or as a parameter name in the parameter type list.

4 Within the lambda expression, identifiers (including explicit and implicit captures, and parameters of the lambda) shall be used according to the usual scoping rules, but outside the assignment expression of a value capture the following exceptions apply to identifiers that are declared in a scope that strictly includes the lambda expression:

— Objects or type definitions with VM type shall not be used.

— Objects with automatic storage duration shall not be evaluated.[114)]

5 ~~The~~ After determining the type of all captures and parameters, either directly or because a type-generic lambda appears in a function-call or conversion to function pointer, the function body shall be such that a return type *type* according to the rules in 6.8.6.4 can be inferred. If the lambda occurs in a conversion to a function pointer, the inferred return type shall be compatible to the specified return type of the function pointer; if additionally the lambda is type-generic, the return type shall be the same as the specified return type.

6 When a lambda expression with an underspecified parameter is evaluated as a void expression, the capture clause shall fulfill the constraints as specified above. The parenthesized parameter list shall provide a valid list of declarations of parameters, only that one or more of these may have an underspecified type. After that shall follow a `{` token, a balanced token sequence (**??**), and a `}` token.[115)]

Semantics

7 The optional attribute specifier sequence in a lambda expression appertains to the resulting lambda value. If the parameter clause is omitted, a clause of the form `()` is assumed. A lambda expression without any capture is called a *function literal expression*, otherwise it is called a *closure expression*. A lambda value originating from a function literal expression is called a *function literal*, otherwise it is called a *closure*.

8 Similar to a function definition, a lambda expression forms a single block scope that comprises its capture clause, its parameter clause and its function body. Each explicit capture and parameter has a scope of visibility that starts immediately after its definition is completed and extends to the end of the function body. The scope of visibility of implicit captures is the function body. In particular,

[114)] Identifiers of visible automatic objects that are not captures and that do not have a VM type, may still be used if they are not evaluated, for example in **`sizeof`** expressions, in **`typeof`** specifiers (if they are not lambdas themselves) or as controlling expression of a generic primary expression.

[115)] That means, besides the validity of the capture clause and the parameter list, an implementation is only required to parse the function body as a token sequence but is not required to diagnose additional constraints, such as the validity of the use of keywords or identifiers within the function body if these are possibly restricted through a syntax derivation or additional constraints.

captures and parameters are visible throughout the whole function body, unless they are redeclared in a depending block within that function body. ~~Captures~~ Value captures and parameters have automatic storage duration; in each function call to the formed lambda value, a new instance of each value capture and parameter is created and initialized in order of declaration and has a lifetime until the end of the call, only that the addresses of captures are not necessarily unique.

9 A lambda expression for which at least one parameter declaration in the parameter list has no type specifier is a *type-generic lambda* with an imcomplete lambda type. It shall only be evaluated as a void expression, be the postfix expression of a function call or, if the capture clause is empty, be the operand of a conversion to a pointer to function with fully specified parameter types, see 6.3.2.1. For a void expression, it has only the side effects that result from the evaluation of the capture clause and shall be syntactically correct as indicated in the constraints; the translation may fail, if the function body is such that no possible function call arguments or target types for a conversion could successfully complete the lambda type; the lambda expression shall otherwise be ignored.

10 For a function call, the type of an argument (after lvalue, array-to-pointer or function-to-pointer conversion) to an underspecified parameter shall be such that it can be used to complete the type of that parameter analogous to 6.7.11, only that the inferred type for an parameter of array or function type is adjusted analogously to function declarators (6.7.6.3) to a possibly qualified object pointer type (for an array) or to a function pointer type (for a function) to match type of the argument. For a conversion of any arguments, the parameter types shall be those of the function type.

11 If a capture `id` is defined without an assignment expression, the assignment expression is assumed to be `id` itself, referring to the object of automatic storage duration of the surrounding scope that exists according to the constraints.[116)]

12 The implicit or explicit assignment expression `E` in the definition of a value capture determines a value E_0 with type T_0, which is `E` after possible lvalue, array-to-pointer or function-to-pointer conversion. The type of the capture is T_0 **`const`** and its value is E_0 for all evaluations in all function calls to the lambda value. If, within the function body, the address of the capture `id` or one of its members is taken, either explicitly by applying a unary & operator or by an array to pointer conversion,[117)] and that address is used to modify the underlying object, the behavior is undefined.

13 The evaluation of the explicit or implicit assignment expressions of value captures takes place during each evaluation of the lambda expression. The evaluation of assignment expressions for explicit value captures is sequenced in order of declaration; an earlier capture may occur within an assignment expression of a later one. The objects of automatic storage duration corresponding to implicit value captures are evaluated unsequenced among each other. The evaluation of a lambda expression is sequenced before any use of the resulting lambda value. For each call to a lambda value, explicit value captures (with type and value as determined during the evaluation of the lambda expression) and then parameter types and values are determined in order of declaration. Explicit value captures and earlier parameters may occur within the declaration of a later one.

14 For each lambda expression, the return type *type* is inferred as indicated in the constraints. A lambda expression λ that is not type-generic has an unspecified lambda type `L` that is the same for every evaluation of λ. ~~As~~ ; as a result of the expression, a value of type `L` is formed that identifies λ and the specific set of values of the identifiers in the capture clause for the evaluation, if any. This is called a *lambda value*. It is unspecified, whether two lambda expressions λ and κ share the same lambda type even if they are lexically equal but appear at different points of the program. Objects of lambda type shall not be modified.

15 A lambda expression λ that is generic has an incomplete lambda type that is completed when the expression is used directly in a function call expression or converted to a function pointer. When used in a function call, the parameter types are inferred in order of declaration, but after the evaluation of the assignment expressions of the explicit value captures, after which the return type of the lambda is inferred from the function body. The so completed lambda value is then used in the function call which is sequenced after the evaluation of the lambda expression.

[116)] The evaluation rules in the next paragraph then stipulate that it is evaluated at the point of evaluation of the lambda expression, and that within the body of the lambda an unmutable **`auto`** object of the same name, value and type is made accesssible.

[117)] The capture does not have array type, but if it has a union or structure type, one of its members may have such a type.

19 **EXAMPLE 3** Consider the following type-generic function literal that computes the maximum value of two parameters `X` and `Y`.

```
#define MAXIMUM(X, Y)                                   \
    [](auto a, auto b){                                 \
      return (a < 0)                                    \
             ? ((b <  0) ? ((a < b) ? b : a) : b)       \
             : ((b >= 0) ? ((a < b) ? b : a) : a);      \
    }(X , Y)

    auto R = MAXIMUM(-1, -1U);
    auto S = MAXIMUM(-1U, -1L);
```

After preprocessing, the definition of `R` becomes

```
auto R = [](auto a, auto b){
  return (a < 0)
     ? ((b <  0) ? ((a < b) ? b : a) : b)
     : ((b >= 0) ? ((a < b) ? b : a) : a);
  }(-1, -1U);
```

To determine type and value of `R`, first the type of the parameters in the function call are inferred to be **`signed int`** and **`unsigned int`**, respectively. With this information, the type of the **`return`** expression becomes the common arithmetic type of the two, which is **`unsigned int`**. Thus the return type of the lambda is that type. The resulting lambda value is the first operand to the function call operator `()`. So `R` has the type **`unsigned int`** and a value of **`UINT_MAX`**.

For `S`, a similar deduction shows that the value still is **`UINT_MAX`** but the type could be **`unsigned int`** (if **`int`** and **`long`** have the same width) or **`long`** (if **`long`** is wider than **`int`**).

As long as they are integers, regardless of the specific type of the arguments, the type of the expression is always such that the mathematical maximum of the values fits. So `MAXIMUM` implements a type-generic maximum macro that is suitable for any combination of integer types.

20 **EXAMPLE 4**

```
void matmult(size_t k, size_t l, size_t m,
             double const A[k][l], double const B[l][m], double const C[k][m]) {
  // dot product with stride of m for B
  // ensure constant propagation of l and m
  auto const λδ = [l,m](double const v[l], double const B[l][m], size_t m0) {
    double ret = 0.0;
    for (size_t i = 0; i < l; ++i) {
      ret += v[i]*B[i][m0];
    }
    return ret;
  };
  // vector matrix product
  // ensure constant propagation of l and m, and accessibility of λδ
  auto const λμ = [l, m, λδ](double const v[l], double const B[l][m], double res[m]) {
    for (size_t m0 = 0; m0 < m; ++m0) {
      res[m0] = λδ(v, B, m0);
    }
  };
  for (size_t k0 = 0; k0 < k; ++k0) {
    double const (*Ap)[l] = A[k0];
    double (*Cp)[m] = C[k0];
    λμ(*Ap, B, *Cp);
  }
}
```

This function evaluates two closures; $\lambda\delta$ has a return type of **`double`**, $\lambda\mu$ of **`void`**. Both lambda values serve repeatedly as first operand to function evaluation but the evaluation of the captures is only done once for each of the closures. For the purpose of optimization, an implementation could generate copies of the underlying functions for each evaluation of such a closure such that the values of the captures `l` and `m` are replaced on a machine instruction level.

```
    }
```

Forward references: function declarators (6.7.6.3), function definitions (6.9.1), initialization (6.7.10).

6.7.6.3 Function declarators (including prototypes)

Constraints

1 A function declarator shall not specify a return type that is a function type or an array type.

2 The only storage-class ~~specifier~~ specifiers that shall occur in a parameter declaration ~~is~~ are **`auto`** and **`register`**.

3 An identifier list in a function declarator that is not part of a definition of that function shall be empty. A parameter declaration without type specifier shall not be formed, unless it includes the storage class specifier **`auto`** and unless it appears in the parameter list of a lambda expression.

4 After adjustment, the parameters in a parameter type list in a function declarator that is part of a definition of that function shall not have incomplete type.

Semantics

5 If, in the declaration "`T D1`", `D1` has the form

`D (` *parameter-type-list* `)`

or

`D (` *identifier-list*$_{\text{opt}}$ `)`

and the type specified for *ident* in the declaration "`T D`" is "*derived-declarator-type-list T*", then the type specified for *ident* is "*derived-declarator-type-list* function returning the unqualified version of *T*".

6 A parameter type list specifies the types of, and may declare identifiers for, the parameters of the function.

7 ~~A~~ After the declared types of all parameters have been determined in order of declaration, any declaration of a parameter as "array of *type*" shall be adjusted to "qualified pointer to *type*", where the type qualifiers (if any) are those specified within the `[` and `]` of the array type derivation. If the keyword **`static`** also appears within the `[` and `]` of the array type derivation, then for each call to the function, the value of the corresponding actual argument shall provide access to the first element of an array with at least as many elements as specified by the size expression.

8 A declaration of a parameter as "function returning *type*" shall be adjusted to "pointer to function returning *type*", as in 6.3.2.1.

9 If the list terminates with an ellipsis (`, ...`), no information about the number or types of the parameters after the comma is supplied.[159)]

10 The special case of an unnamed parameter of type **`void`** as the only item in the list specifies that the function has no parameters.

11 If, in a parameter declaration, an identifier can be treated either as a typedef name or as a parameter name, it shall be taken as a typedef name.

12 If the function declarator is not part of a definition of that function, parameters may have incomplete type and may use the `[*]` notation in their sequences of declarator specifiers to specify variable length array types.

13 The storage-class specifier in the declaration specifiers for a parameter declaration, if present, is ignored unless the declared parameter is one of the members of the parameter type list for a function definition.

14 An identifier list declares only the identifiers of the parameters of the function. An empty list in a function declarator that is part of a definition of that function specifies that the function has no parameters. The empty list in a function declarator that is not part of a definition of that function

[159)] The macros defined in the `<stdarg.h>` header (7.16) can be used to access arguments that correspond to the ellipsis.

of the same rank and signedness but that are nevertheless different types shall not be considered.[171] If the assignment-expression is the evaluation of a bit-field designator, the inferred type shall be the standard integer type that would be chosen by a generic primary expression with the that bit-field as controlling expression. If *type* is a VM type, the variable array bounds shall be such that the declared types for all defined objects and their assignment expression correspond as required for all possible executions of the current function. If the assignment expression has lambda type, the lambda type shall be complete, the declaration shall only define one object and shall only consist of storage class specifiers, qualifiers, the identifier that is to be declared, and the initializer.

Description

6 Although there is no syntax derivation to form declarators of lambda type, values of lambda type can be used as assignment expression and the inferred type is that lambda type, possibly qualified. Otherwise, provided the constraints above are respected, in an underspecified declaration the type of the declared identifiers is the type after the declaration would have been adjusted by a choice for *type* as described. If the declaration is also an object definition, the assignment expressions that are used to determine types and initial values of the objects are evaluated at most once; the scope rules as described in 6.2.1 then also prohibit the use of the identifier of an object within the assignment expression that determines its type and initial value.

7 **NOTE 1** Because of the relatively complex syntax and semantics of type specifiers, the requirements for *type* use a **`typeof`** specifier. If for example the identifier or tag name of the type of the initializer expression `v` in the initializer of `x` is shadowed

```
auto x = v;
```

a type *type* as required can still be found and the definition can be adjusted as follows:

```
typeof(v) x = v;
```

Such a possible adjustment not withstanding, if `v` is a VM type, the requirements ensure that `v` is evaluated at most once.

8 **NOTE 2** The scope of the identifier for which the type is inferred only starts after the end of the initializer (6.2.1), so the assignment expression cannot use the identifier to refer to the object or function that is declared, for example to take its address. Any use of the identifier in the initializer is invalid, even if an entity with the same name exists in an outer scope.

```
{
  double a = 7;
  double b = 9;
  {
    double b = b * b;    // error, RHS uses uninitialized variable
    printf("%g\n", a);   // valid, uses "a" from outer scope, prints 7
    auto a   = a * a;    // error, "a" from outer scope is already shadowed
  }
  {
    auto b   = a * a;    // valid, uses "a" from outer scope
    auto a   = b;        // valid, shadows "a" from outer scope
    ...
    printf("%g\n", a);   // valid, uses "a" from inner scope, prints 49
  }
  ...
}
```

9 **NOTE 3** Declarations that are the definition of several objects, may make type inferrence difficult and not portable.

```
enum A { aVal, } aObj = aVal;
enum B { bVal, } bObj = bVal;
int  au = aObj, bu = bObj;  // valid, values have type compatible to int
auto ax = aObj, bx = bObj;  // invalid, same rank but different types
auto ay = aObj;             // valid, ay has type enum A
auto by = bObj;             // valid, by has type enum B
auto az = aVal, bz = bVal;  // valid, az and bz have type int
```

[171] This can for example be two different enumerated types that are compatible to the same basic type. Note nevertheless, that enumeration constants have type **`int`**, so using these will never lead to the inference of an enumerated type.